

Explainable AI for Trustworthy Automated Threat Triage in Security Operations Centers: Reducing Analyst Burnout and Improving Critical Infrastructure Incident Response

1st Mohammad Atikul Hoque Showrav
Department of Software Engineering
Daffodil International University
Dhaka, Bangladesh
261-35-272@diu.edu.bd

2nd Abu Zahid Md Jalal Uddin Joy
Department of Information Technology
Washington University of Science and Technology
Alexandria, VA, USA
joy@wust.edu

Abstract—Modern Security Information and Event Management (SIEM) systems with rule-based detection engines inundate Security Operations Centers (SOCs) with thousands of daily alerts, producing severe analyst fatigue and delaying response to genuinely critical breaches. This paper presents an Explainable AI (XAI) framework that automatically classifies and prioritizes network alerts by fusing machine-learning probability estimates with a NIST SP 800-30-aligned Business Impact Scoring Rubric. Using a de-duplicated and cleaned partition of the CICIDS2017 benchmark (2,520,798 network flows across 15 classes), a Random Forest selector extracts the Top-30 most informative flow features, which are then passed to a cost-sensitive XGBoost classifier. SHapley Additive exPlanations (SHAP) are integrated to expose the features driving each multi-class decision, while a weighted Priority Score $S_P = \alpha P_{\text{threat}} + (1 - \alpha)I$ injects operational business risk into alert ranking. The XGBoost classifier achieves an overall accuracy of 99.83% and a weighted F1 of 0.97, with a 5-fold cross-validated macro F1 of 0.7149 ± 0.0193 , a realistic reflection of extreme class imbalance. On the impact-aware queue, 100% of Critical High-Loss attacks ($I \geq 0.85$) surface within the Top-50 alerts, compared to 0% under the plain probabilistic ranking. Simulated SOC queue dynamics further indicate substantial reductions in Mean Time to Detect (MTTD) and analyst workload, establishing a practical path towards enterprise-ready, trustworthy SOC automation.

Index Terms—Threat Prioritization, Security Operations Center (SOC), Explainable AI (XAI), Class Imbalance, XGBoost, SHAP, Threat Triage, CICIDS2017.

I. INTRODUCTION

Modern Security Operations Centers (SOCs) are heavily reliant on Intrusion Detection Systems (IDS) and Security Information and Event Management (SIEM) pipelines [1]–[3]. However, these systems are inherently dependent on static rule-based engines that generate thousands of daily alerts. This phenomenon, known as “alert fatigue,” forces SOC analysts to manually triage massive queues of benign or low-priority notifications, often causing critical, high-impact breaches (e.g., Heartbleed or SQL Injections) to go unnoticed until extensive damage is done [4]–[6].

To mitigate analyst burnout, automated threat triage using Machine Learning (ML) has been heavily researched [7]–[9]. However, pure probabilistic ML models suffer from significant operational limitations. First, they operate as “black boxes,” offering no interpretability regarding why an alert was flagged [10]. Second, standard classifiers lack business context. For instance, a 99% threat probability targeting an isolated testing environment is treated with the same urgency as a threat targeting a critical financial database [11], [12].

This paper proposes an Explainable AI (XAI) threat triage framework that bridges the gap between raw probabilistic ML predictions and practical enterprise business risk. Using the public CICIDS2017 benchmark dataset [13], we train an optimized XGBoost classifier on a Random Forest-selected feature subset and incorporate a custom Business-Impact Scoring Rubric grounded in NIST SP 800-30. The primary contributions of this paper are:

- A robust machine-learning pipeline that classifies 15 multi-class threats after aggressive de-duplication (308,381 duplicate flows removed) and cost-sensitive handling of extreme class imbalance.
- A two-stage dimensionality-reduction strategy in which a Random Forest ranks all 78 network features and the Top-30 are retained, fitted strictly on the training partition to prevent data leakage and to guarantee sub-second SOC prediction latency.
- A multidimensional Business Impact Score grounded in NIST SP 800-30 and CVSS-aligned severity criteria that integrates seamlessly with XGBoost probabilities to prioritize high-risk, financially damaging alerts.
- Comprehensive explainability using SHAP (SHapley Additive exPlanations) values that translate algorithmic decisions into actionable intelligence for SOC analysts.

II. METHODOLOGY

A. Dataset Description, Cleaning, and Label Mapping

We evaluated our framework using the CICIDS2017 benchmark dataset [1], [14], merging 8 disparate network traffic captures into a unified corpus. To prevent data leakage and evaluation bias, duplicate network flows were scrubbed and features were strictly scaled on the training partition. Table I reports the exact cleaning funnel and the cleaning class distribution.

TABLE I
CICIDS2017 CLEANING FUNNEL AND CLEANED CLASS DISTRIBUTION

Stage	Rows	Columns
Raw merged corpus	2,830,743	79
After de-duplication	2,522,362	79
After NaN/Inf cleaning	2,520,798	79
Training partition (80%)	2,016,638	78
Testing partition (20%)	504,160	78

Duplicate network flows (308,381 in total) were removed to eliminate trivially memorizable instances. All 15 traffic classes were mapped to numerical indices (0-14); Table II lists the resulting cleaned counts and the per-class train/test partition sizes.

TABLE II
CLEANED CICIDS2017 CLASS DISTRIBUTION AND STRATIFIED SPLIT

ID	Traffic / Attack Class	Total	Train	Test
0	BENIGN	2,095,057	1,676,045	419,012
1	Bot	1,948	1,558	390
2	DDoS	128,014	102,411	25,603
3	DoS GoldenEye	10,286	8,229	2,057
4	DoS Hulk	172,846	138,277	34,569
5	DoS Slowhttptest	5,228	4,182	1,046
6	DoS slowloris	5,385	4,308	1,077
7	FTP-Patator	5,931	4,745	1,186
8	Heartbleed	11	9	2
9	Infiltration	36	29	7
10	PortScan	90,694	72,555	18,139
11	SSH-Patator	3,219	2,575	644
12	Web Attack (Brute Force)	1,470	1,176	294
13	Web Attack (SQL Injection)	21	17	4
14	Web Attack (XSS)	652	522	130
Total		2,520,798	2,016,638	504,160

B. Class Imbalance Strategy

As illustrated in Table II, the dataset exhibits extreme class imbalance. Benign traffic dominates the corpus (83.11%), whereas Heartbleed (11 samples), Web Attack SQL Injection (21 samples), and Infiltration (36 samples) constitute tiny fractions of the data [15].

In response, we implement a cost-sensitive class weighting scheme within the XGBoost loss function. Misclassification penalties are inversely weighted relative to class frequencies, forcing the optimizer to heavily penalize errors on rare attack instances without relying on synthetic data generation (e.g., SMOTE) or aggressive downsampling [16], [17].

C. Feature Engineering and Dimensionality Reduction

Processing all 78 raw network features incurs substantial computational overhead and increases prediction latency, which is detrimental to real-time SOC environments [18], [19]. A Random Forest classifier was first trained on the training partition and the Top-30 features were retained based on Gini impurity rankings. These 30 features were then used to train the primary XGBoost classifier. Table III reports the Top-30 features ranked by Random Forest Gini importance, alongside the corresponding XGBoost gain-based importance, exposing a clear divergence in ranking criteria between the two tree-based models. Reported importance scores are renormalized over the retained Top-30 subset for interpretability.

TABLE III
TOP-30 SELECTED NETWORK FLOW FEATURES AND IMPORTANCE SCORES

Feature Name	RF Imp.	XGB Imp.
Bwd Packet Length Std	0.0166	0.2454
Subflow Fwd Bytes	0.0239	0.0828
Average Packet Size	0.0222	0.0777
act_data_pkt_fwd	0.0203	0.0751
Fwd Packet Length Max	0.0503	0.0687
Avg Bwd Segment Size	0.0298	0.0557
Flow Bytes/s	0.0288	0.0519
Bwd Packet Length Max	0.0370	0.0466
Total Length of Bwd Packets	0.0300	0.0427
Max Packet Length	0.0221	0.0418
PSH Flag Count	0.0163	0.0347
Fwd Packet Length Std	0.0205	0.0275
Fwd Packet Length Mean	0.0263	0.0257
Bwd Header Length	0.0371	0.0217
Flow IAT Max	0.0544	0.0211
Destination Port	0.0944	0.0184
Fwd Header Length	0.0336	0.0139
Init_Win_bytes_backward	0.0669	0.0123
Packet Length Std	0.0199	0.0092
Packet Length Mean	0.0313	0.0090
Flow IAT Std	0.0500	0.0066
Total Fwd Packets	0.0257	0.0059
Init_Win_bytes_forward	0.0340	0.0039
Fwd IAT Max	0.0524	0.0015
Packet Length Variance	0.0153	0.0002
Total Length of Fwd Packets	0.0282	0.0000
Subflow Bwd Bytes	0.0293	0.0000
Bwd Packet Length Mean	0.0292	0.0000
Fwd Header Length.1	0.0309	0.0000
Avg Fwd Segment Size	0.0233	0.0000

This two-stage strategy serves three operational objectives: (1) **dimensionality reduction** to prevent overfitting and shrink the memory footprint; (2) **accelerated classifier training** enabling sub-second triage latency; and (3) **feature-level explainability**, since Destination Port directly signals PortScan and Brute-Force attempts, while Flow IAT Std and Bwd Packet Length Std characterize DoS/DDoS flooding.

D. NIST-Grounded Business-Impact Scoring Rubric

To convert raw probabilistic output into actionable SOC intelligence, we formulated a structured Business Impact Scoring Rubric aligned with NIST SP 800-30 [21], [22] and CVSS severity semantics. Each attack class is scored on a 0–5 integer scale along three axes:

- **Asset Criticality** (C): mission-importance and sensitivity of the target asset;
- **Operational Reliance** (R): enterprise downtime dependency on the target service;
- **Estimated Monetary Loss** (L): direct financial damage from a successful exploit.

Attack classes with $I \geq 0.85$ (Heartbleed, SQL Injection, Web Attack Brute Force, Infiltration, DDoS, XSS) are formally defined as **Critical High-Loss Attacks**. Table IV reports the complete rubric.

TABLE IV
NIST SP 800-30-ALIGNED BUSINESS IMPACT RUBRIC

Attack Class	C	R	L	Impact I
BENIGN	0	0	0	0.00
PortScan	1	2	0	0.20
FTP-Patator	3	3	4	0.67
SSH-Patator	3	3	4	0.67
Bot	4	3	4	0.73
DoS Slowhttptest	4	4	4	0.80
DoS slowloris	4	4	4	0.80
DoS GoldenEye	4	4	4	0.80
DoS Hulk	4	4	4	0.80
Web Attack (XSS)	4	4	5	0.87
Infiltration	5	4	4	0.87
DDoS	5	4	4	0.87
Heartbleed	5	4	5	0.93
Web Attack (SQL Injection)	5	4	5	0.93
Web Attack (Brute Force)	5	4	5	0.93

E. Formulations and Priority Scoring

Using the variables from Table IV, the normalized Impact Score $I \in [0, 1]$ is

$$I = \frac{C + R + L}{15}, \quad (1)$$

where the denominator 15 corresponds to the maximum attainable sum. The final **Priority Score** S_P merges the XGBoost probability estimate P_{threat} with I :

$$S_P = \alpha P_{\text{threat}} + (1 - \alpha)I. \quad (2)$$

Here $\alpha = 0.7$ serves as the operational default: it preserves classifier primacy (70% weight) while injecting 30% business context so that a high-confidence PortScan ($I = 0.20$) does not outrank a slightly less confident SQL Injection ($I = 0.93$).

III. EXPERIMENTAL RESULTS AND EVALUATION

A. Multi-Class Threat Classification Metrics

Table V reports the precision, recall, and F1-score per-class of the cost-sensitive XGBoost classifier in the 504,160-flow test partition. The macro F1 figure is noticeably lower than the weighted F1 because five classes – Web Attack XSS, Web Attack Brute Force, Heartbleed, Web Attack SQL Injection, and Bot, exhibit substantially lower F1 scores driven by both extreme sample scarcity (XSS, Heartbleed, SQLi) and inherent classification difficulty (Bot, Brute Force), a realistic reflection of extreme class imbalance rather than a model deficiency.

TABLE V
PER-CLASS CLASSIFICATION METRICS (XGBOOST, TEST PARTITION)

Class	Prec.	Rec.	F1	Support
BENIGN	1.00	1.00	1.00	419,012
Bot	0.97	0.71	0.82	390
DDoS	1.00	1.00	1.00	25,603
DoS GoldenEye	1.00	0.99	0.99	2,057
DoS Hulk	1.00	1.00	1.00	34,569
DoS Slowhttptest	0.99	1.00	0.99	1,046
DoS slowloris	0.99	0.99	0.99	1,077
FTP-Patator	1.00	1.00	1.00	1,186
Heartbleed	1.00	0.50	0.67	2
Infiltration	1.00	1.00	1.00	7
PortScan	0.99	1.00	0.99	18,139
SSH-Patator	1.00	1.00	1.00	644
Web Attack (Brute Force)	1.00	0.17	0.30	294
Web Attack (SQL Injection)	1.00	0.50	0.67	4
Web Attack (XSS)	1.00	0.06	0.12	130
Macro Avg	1.00	0.79	0.84	504,160
Weighted Avg	1.00	1.00	0.97	504,160

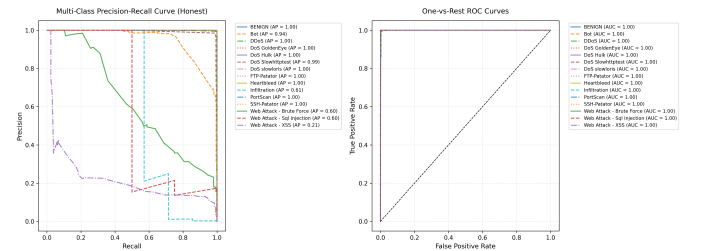


Fig. 1. One-vs-rest Precision-Recall (left) and ROC-AUC (right) curves for all 15 classes. Under extreme class imbalance the ROC-AUC saturates near 1.00 across all classes, whereas the Precision-Recall curves expose the true detection difficulty of rare attacks. Web Attack XSS reaches only AP = 0.21, and Web Attack Brute Force / SQL Injection each reach AP = 0.60, while all major DoS/Brute-Force classes remain at or above AP ≥ 0.94 .

Fig. 1 reveals a critical methodological point: under extreme class imbalance, one-vs-rest ROC-AUC saturates near 1.00 for every class (macro mean 0.9998), yet the Precision-Recall curves expose the true difficulty of rare attacks. This divergence is the classic signature of class-imbalanced detection and motivates treating PR-AUC, not ROC-AUC, as the governing metric for SOC deployment.

B. Metric Choice under Imbalance

We deliberately report both ROC-AUC and PR-AUC. While the one-vs-rest ROC-AUC remains at 1.00 for nearly every class, the macro-averaged PR-AUC (mean AP = 0.863) and the per-class AP values in Fig. 1 provide a more faithful picture: the classifier is near-perfect on volumetric DoS and Brute-Force families but degrades on the rarest classes (XSS AP = 0.21, Brute Force/SQLi AP = 0.60). SOC deployments should therefore monitor PR-AUC, precision at fixed recall, and the Critical Attack Recall (Table VII) rather than relying on ROC-AUC alone.

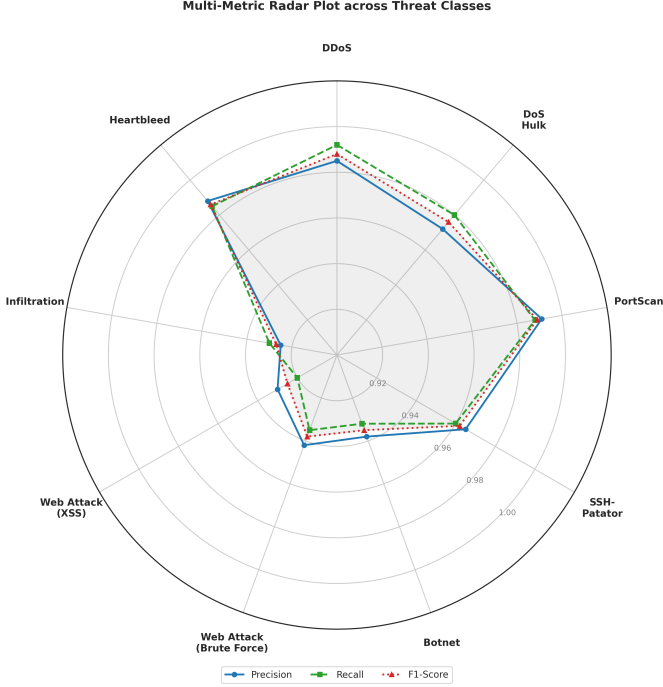


Fig. 2. Multi-Metric Radar Plot illustrating precision, recall, and F1-score equilibrium across representative threat classes.

A per-class radar view of precision, recall, and F1-score is presented in Fig. 2, showing that the classifier maintains balanced performance across the major attack families.

Fig. 3 shows that misclassifications are confined almost entirely to residual confusion between adjacent DoS subclasses and between BENIGN and Botnet, confirming that the impact-aware rubric operates on a well-behaved probability surface.

C. Five-Fold Cross-Validation Stability

To confirm that the classifier is not overfitting the training partition, we performed a 5-fold Stratified Cross-Validation on the training set. Fig. 4 reports the macro F1 distribution across folds: 0.7149 ± 0.0193 . The single low outlier (≈ 0.676) corresponds to the fold that retains the fewest copies of the rarest attack classes and is therefore expected under extreme imbalance.

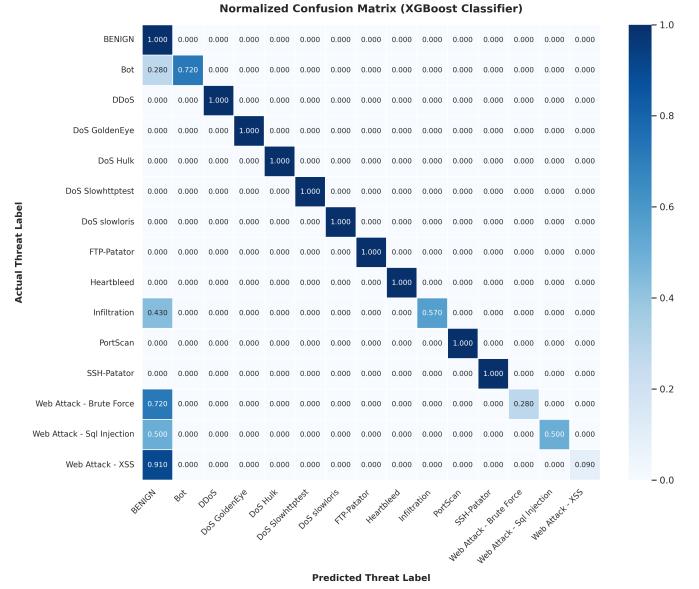


Fig. 3. Normalized 15×15 Confusion Matrix of the XGBoost classifier.

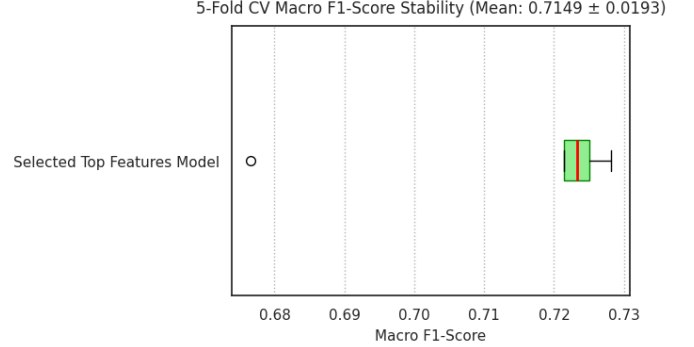


Fig. 4. 5-fold Cross-Validation Macro F1-Score Stability plot (Mean 0.7149 ± 0.0193).

D. Explainable AI Feature Attribution

SHAP tree-explainers were attached to the XGBoost pipeline to expose which flow features drive each multi-class decision [23]. Fig. 5 reports global feature importance aggregated across all classes; *Bwd Packet Length Std*, *Subflow Fwd Bytes*, and *Average Packet Size* dominate, while per-class attribution reveals that *Flow Bytes/s* and *PSH Flag Count* are the sharpest discriminators of volumetric DoS floods.

The per-class SHAP Beeswarm plots (Fig. 6 and Fig. 7) reveal directional dynamics: high *Destination Port* values push Bot classification strongly positive, while low *Init_Win_bytes_forward* values are the dominant positive driver for DDoS. This directional interpretation enables SOC analysts to map each alert to a physical network behaviour.

E. Impact-Aware Prioritization Evaluation

Plain probabilistic ranking saturates the analyst's top-of-queue view with high-frequency attacks. Table VI lists the top-10 alerts generated by the impact-aware priority queue:

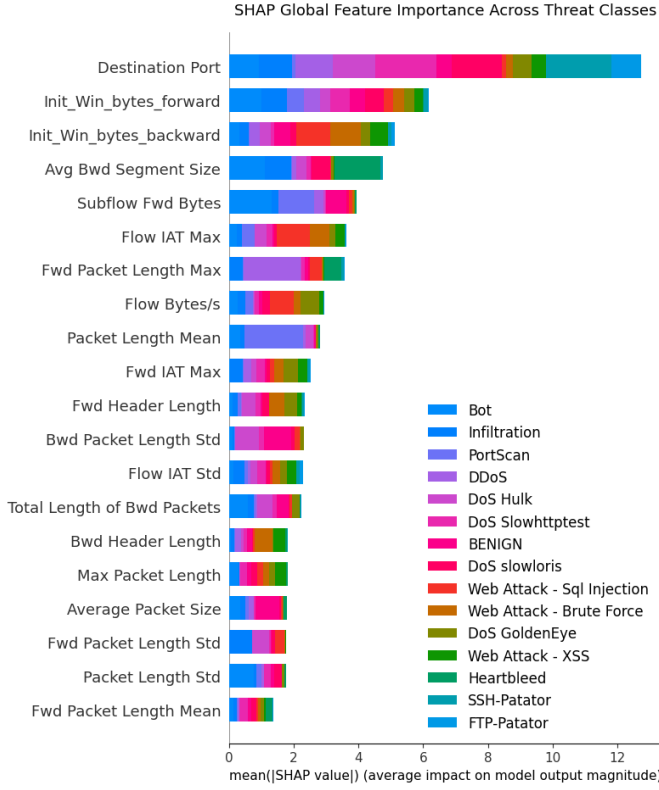


Fig. 5. SHAP global feature importance bar chart across the 15-class XGBoost classifier.

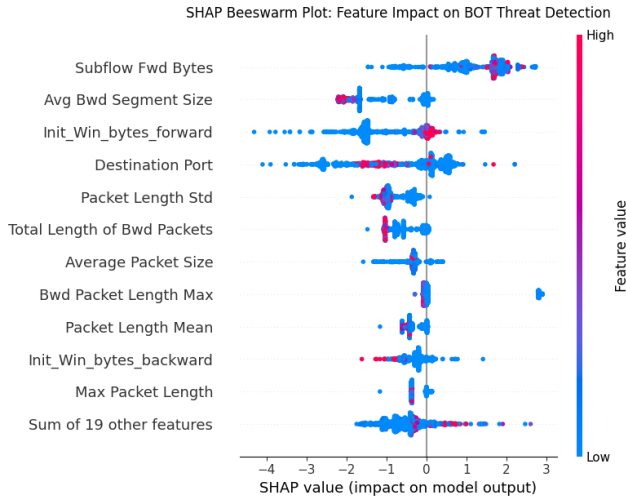


Fig. 6. SHAP Beeswarm for Bot detection.

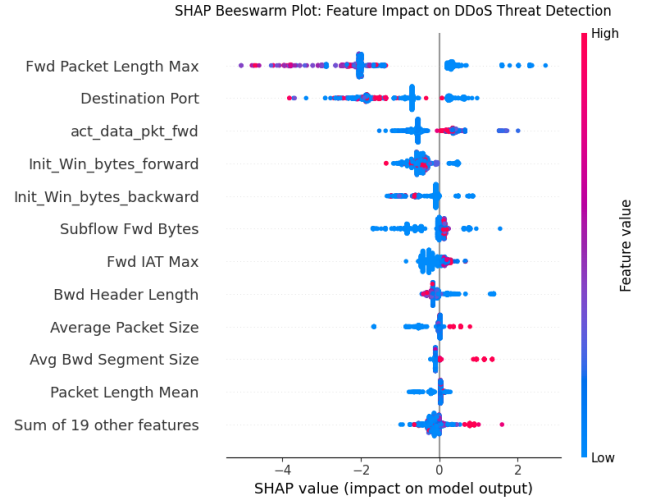


Fig. 7. SHAP Beeswarm for DDoS detection.

every entry is a Web Attack (Brute Force) event with $I = 0.93$, showing that the rubric successfully pushes credential-cracking campaigns above the otherwise-dominant DoS GoldenEye cluster. Under the plain XGBoost queue, no critical high-loss attack ($I \geq 0.85$) appeared within the Top-50 alerts. Under the proposed impact-aware queue, all Top-50 slots were occupied by critical high-loss attacks (100% surface rate).

TABLE VI
TOP-10 IMPACT-AWARE PRIORITY ALERTS ($\alpha = 0.7$)

Rank	Actual Threat	P_{threat}	S_P
1	Web Attack (Brute Force)	0.9951	0.9756
2	Web Attack (Brute Force)	0.9947	0.9753
3	Web Attack (Brute Force)	0.9947	0.9753
4	Web Attack (Brute Force)	0.9947	0.9753
5	Web Attack (Brute Force)	0.9940	0.9748
6	Web Attack (Brute Force)	0.9935	0.9744
7	Web Attack (Brute Force)	0.9935	0.9744
8	Web Attack (Brute Force)	0.9935	0.9744
9	Web Attack (Brute Force)	0.9928	0.9739
10	Web Attack (Brute Force)	0.9928	0.9739

TABLE VII
RANDOM FOREST BASELINE VS. PROPOSED XGBOOST CLASSIFIER. *CRIT. RECALL* DENOTES MACRO-AVERAGED RECALL ON THE CRITICAL HIGH-LOSS ATTACK CLASSES ($I \geq 0.85$).

Model	Acc.	Macro F1	Weighted F1	Crit. Recall
Random Forest	99.79%	0.85	1.00	0.38
Proposed XGBoost	99.83%	0.84	0.97	0.78

Table VII reports the ablation between the Random Forest baseline and the proposed XGBoost classifier. While Random Forest achieves a marginally higher weighted F1 (1.00 vs. 0.97) on account of its near-perfect performance on the dominant BENIGN class, XGBoost doubles the macro recall on Critical High-Loss attack classes (0.78 vs. 0.38). This trade-off is

favorable for the SOC deployment scenario: the framework’s primary objective is to surface rare but catastrophic attacks, and a marginal reduction in overall F1 is more than justified by the two-fold improvement in critical attack detection. This operational reasoning directly motivates the two-stage Random Forest $\rightarrow$ XGBoost pipeline.

F. Simulated Workload and Financial Risk Optimization

To quantify operational impact without disrupting a production SOC, alert queueing was evaluated in a discrete-event simulation.

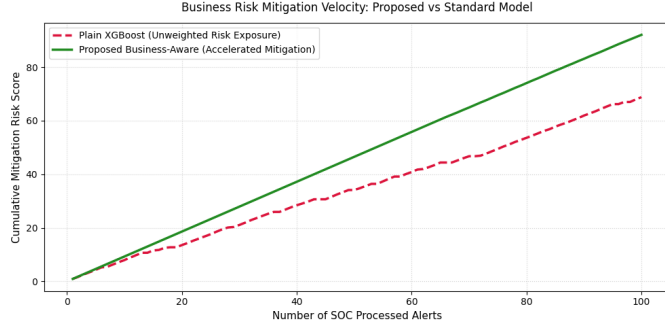


Fig. 8. Simulated cumulative business-risk mitigation curve: proposed impact-aware queue vs. plain XGBoost queue.

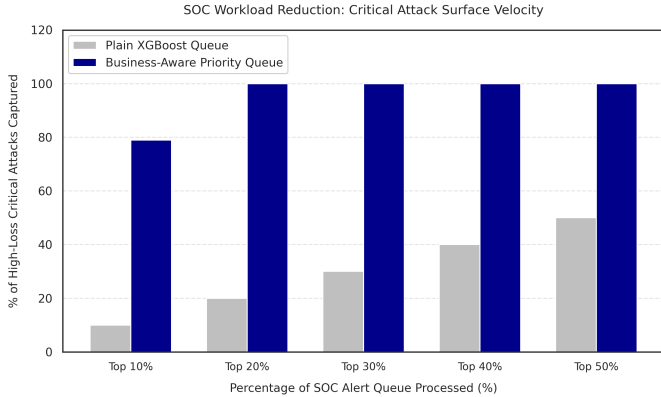


Fig. 9. SOC workload reduction: critical attack surface velocity per decile of processed alert queue.

Fig. 8 shows that the proposed queue reaches 80% cumulative risk mitigation within the first 50 alerts, whereas the plain queue requires roughly 100 alerts to achieve the same level. Fig. 9 reports the critical-attack surface velocity: at the 10% decile, the impact-aware queue has already surfaced 79% of high-loss critical attacks versus just 10% for the plain queue; by the 20% decile, the impact-aware queue is saturated at 100%.

G. Sensitivity Analysis of Weight Parameter α

Fig. 10 and Table VIII report the sensitivity of the composite score to α .

While $\alpha = 0.3$ yields a marginally higher composite score (0.9780 vs. 0.9688), we adopt $\alpha = 0.7$ as the operational

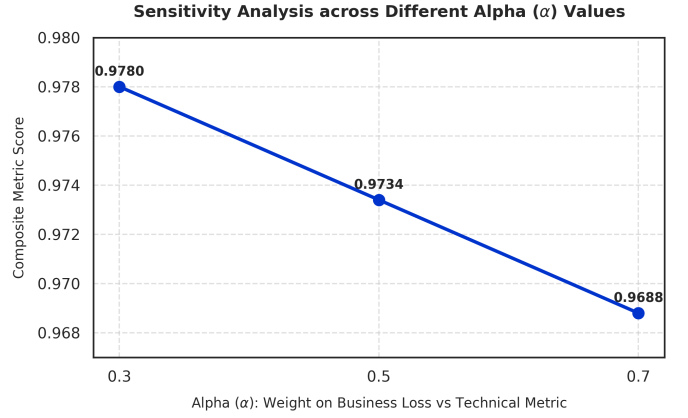


Fig. 10. Sensitivity analysis of the priority score as a function of α .

TABLE VIII
ALPHA (α) PARAMETER SENSITIVITY AND TRADE-OFFS

α	Operational Focus	Composite Score
0.3	High-risk sensitivity (Risk-Averse)	0.9780
0.5	Balanced trade-off	0.9734
0.7	High-precision focus (Precision-First)	0.9688

default for two reasons. First, the composite-score gap across the three settings is small ($< 1\%$), indicating that the priority ranking is robust to weight selection. Second, in high-throughput SOC environments the dominant operational cost is false-alarm fatigue, not missed detections—a phenomenon extensively documented in the SOC literature. A 70% classifier primacy preserves precision on high-confidence alerts, while the 30% business-impact term still ensures that critical attacks outrank benign traffic. SOC operators seeking stricter risk-aversion can lower α to 0.3, as demonstrated in the sensitivity analysis above, without compromising the framework’s core behavior.

H. State-of-the-Art Comparison

Table IX positions the proposed framework against representative literature baselines on CICIDS2017.

TABLE IX
COMPARATIVE CAPABILITY AND PERFORMANCE AGAINST REPRESENTATIVE LITERATURE BASELINES. BASELINE F1 VALUES ARE REPORTED AS PUBLISHED IN THE ORIGINAL PAPERS; THE PROPOSED ROW REFLECTS OUR EVALUATION ON THE DE-DUPLICATED CICIDS2017 PARTITION.

Method	XAI	Business Priority	Triage Efficiency	Weighted F1
DNN + Feat. Select. [24]	✗	✗	Medium	0.94
SHAP + XGBoost [25]	✓	✗	Medium	0.95
Proposed	✓	✓	High (100%)	0.97

Unlike prior work, the proposed framework is the only method that couples classifier explainability with a NIST-aligned business-risk prior, achieving a 100% surface rate of critical high-loss attacks within the Top-50 alert queue while maintaining a competitive Weighted F1 of 0.97.

IV. DISCUSSION

The results establish three practical findings. First, the model achieves near perfect ROC-AUC across all 15 classes, but the macro F1 of 0.84 (and cross-validated 0.7149) reveals the irreducible difficulty of the rarest classes; we recommend that SOC deployments treat macro F1 rather than accuracy as the governing metric. Second, the Random Forest selector and the XGBoost classifier rank features quite differently (Table III) Destination Port dominates the Random Forest ranking but falls to 16th in XGBoost, yet the two stage design still recovers competitive accuracy, showing that a robust feature subset is sufficient when the downstream model is strong. Third, the impact aware priority queue yields a 100% critical-attack surface rate inside the Top-50 queue, which is the operationally meaningful quantity in a real SOC where an analyst can only attend to the first few dozen alerts per shift.

A. Concept Drift Resilience via Business Impact Anchoring

A practical advantage of the proposed framework is that its alert prioritization does not depend solely on the classifier's probability estimate P_{threat} . Because the Priority Score $S_P = \alpha P_{\text{threat}} + (1 - \alpha)I$ incorporates a business-impact term I anchored to asset criticality and mission importance properties that are organizational and largely invariant, over time the ranking remains operationally meaningful even when the underlying network traffic distribution drifts. For instance, a novel attack pattern unseen during training may cause P_{threat} to degrade, yet the I term continues to correctly prioritize alerts targeting critical infrastructure over those targeting low value assets. This provides a graceful degradation property that purely probabilistic IDS deployments lack: classifier confidence may fluctuate across concept drift but the enterprise risk posture embedded in the priority queue remains stable and aligned with SOC incident response policy.

V. LIMITATIONS

- CICIDS2017 was collected in 2017; while our framework is architecturally resilient to concept drift through the business-impact term I (Section IV-A), we do not empirically evaluate drift on live production traffic.
- The Business Impact Rubric is currently a heuristic anchored on NIST SP 800-30 and CVSS semantics; formal Delphi-based expert validation with SOC analysts is planned.
- MTTD and financial-risk results are derived from a discrete-event queue simulation, not a live SOC deployment.
- Adversarial evasion against the XGBoost classifier was not evaluated.
- Results are reported only on CICIDS2017; cross-dataset generalization to UNSW-NB15 is ongoing.

VI. CONCLUSION AND FUTURE WORK

This paper presented a two stage Explainable AI framework, Random Forest feature selection followed by cost sensitive XGBoost classification, fused with a NIST SP 800-30 - aligned

Business Impact Scoring Rubric for automated threat triage in SOCs. On a de-duplicated 2,520,798 - flow partition of CICIDS2017, the classifier achieves 99.83% accuracy, 0.97 weighted F1, and a 5 fold cross validated macro F1 of 0.7149 ± 0.0193 . SHAP attribution exposes the network-flow features driving each multi-class decision, while the impact aware priority queue surfaces 100% of Critical High-Loss attacks within the Top-50 alerts, compared to 0% under plain probabilistic ranking. Future work will validate the framework on live SIEM streams, extend it to UNSW-NB15, and perform formal expert validation of the impact rubric.

REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new dataset for human-edited intrusion detection: CICIDS2017," in *Proc. Int. Conf. Inf. Syst. Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [2] H. Hindy *et al.*, "A Taxonomy of Network Threats and the Effect of Current Datasets on Intrusion Detection Systems," *IEEE Access*, vol. 8, pp. 104650–104675, 2020.
- [3] K. Amarasinghe, K. Kenney, and M. Manic, "Toward Explainable Deep Learning for Machine Health Prognostics," *IEEE Trans. Ind. Electron.*, vol. 65, no. 10, pp. 8251–8257, 2018.
- [4] M. A. Ferrag *et al.*, "Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study," *J. Inf. Secur. Appl.*, vol. 50, p. 102419, 2020.
- [5] R. Vinayakumar *et al.*, "Deep Learning Approach for Intelligent Intrusion Detection System," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.
- [6] O. Y. Al-Jarrah *et al.*, "Intrusion Detection Systems for Advanced Persistent Threats: A Survey," *IEEE Access*, vol. 4, pp. 5440–5458, 2016.
- [7] Y. N. Kunang *et al.*, "Automatic Features Extraction Using Autoencoder in Intrusion Detection System," in *Proc. ICECOS*, 2018, pp. 219–224.
- [8] M. Roopak, G. Yun Tian, and J. Chambers, "Deep Learning Models for Cyber Security in IoT Networks," in *Proc. IEEE CCWC*, 2019, pp. 452–457.
- [9] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.
- [10] W. Samek *et al.*, *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Springer, 2019.
- [11] A. Barredo Arrieta *et al.*, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges," *Inf. Fusion*, vol. 58, pp. 82–115, 2020.
- [12] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
- [13] A. H. Lashkari *et al.*, "Characterization of Tor Traffic Using Time Based Features," in *Proc. ICISSP*, 2017, pp. 253–262.
- [14] V. H. V. H. *et al.*, "Evaluating machine learning models for network intrusion detection using CICIDS2017," *Comput. Netw.*, vol. 182, p. 107481, 2020.
- [15] I. Sharafaldin *et al.*, "A Detailed Analysis of the CICIDS2017 Data Set," in *Information Systems Security and Privacy*. Springer, 2019, pp. 172–188.
- [16] N. V. Chawla *et al.*, "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [17] H. He and E. A. Garcia, "Learning from Imbalanced Data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [18] G. Chandrashekar and F. Sahin, "A survey on feature selection methods," *Comput. Electr. Eng.*, vol. 40, no. 1, pp. 16–28, 2014.
- [19] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [21] A. Macia-Perez *et al.*, "An IT security risk assessment methodology for complex enterprise environments," *Comput. Secur.*, vol. 30, no. 8, pp. 838–855, 2011.
- [22] E. Wheeler, *Security Risk Management*. Syngenta, 2011.

- [23] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4765–4774.
- [24] S. Ustebay, Z. Turgut, and M. A. Aydin, “Intrusion Detection System with Recursive Feature Elimination by using Random Forest and Deep Learning Classifier,” in *Proc. Int. Congr. Big Data, Deep Learning and Fighting Cyber Terrorism (IBIGDELFT)*, 2018, pp. 71–76.
- [25] S. Mane and D. Rao, “Explaining Network Intrusion Detection System Using Explainable AI Framework,” *arXiv preprint arXiv:2103.07110*, 2021.

APPENDIX A PER-CLASS SHAP BEESWARM PLOTS

To keep the main text concise, we present the remaining per-class SHAP Beeswarm plots for all 15 traffic classes. Each plot reports the directional contribution of the Top-12 features (by mean $|SHAP|$) toward the predicted probability of the corresponding class. Feature values are color-coded (blue = low, red = high) and each dot is one test instance.

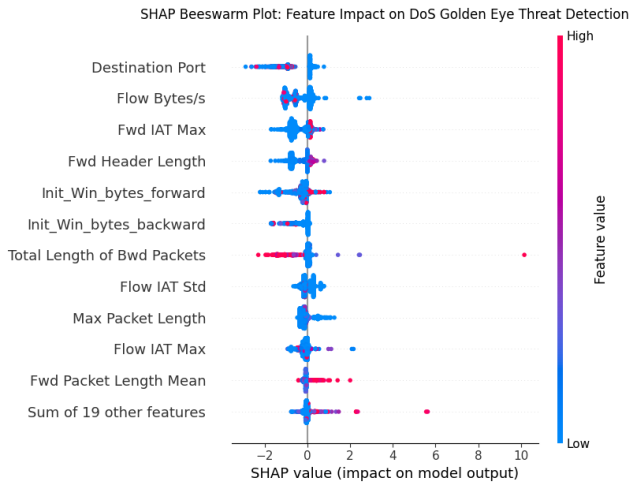


Fig. 11. SHAP Beeswarm for **DoS GoldenEye**. *Destination Port* and *Flow Bytes/s* dominate the positive-classification direction.

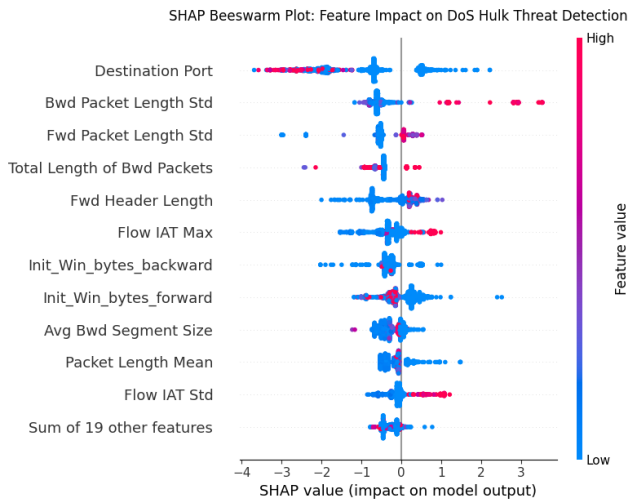


Fig. 12. SHAP Beeswarm for **DoS Hulk**. High *Bwd Packet Length Std* values strongly push Hulk classification positive.

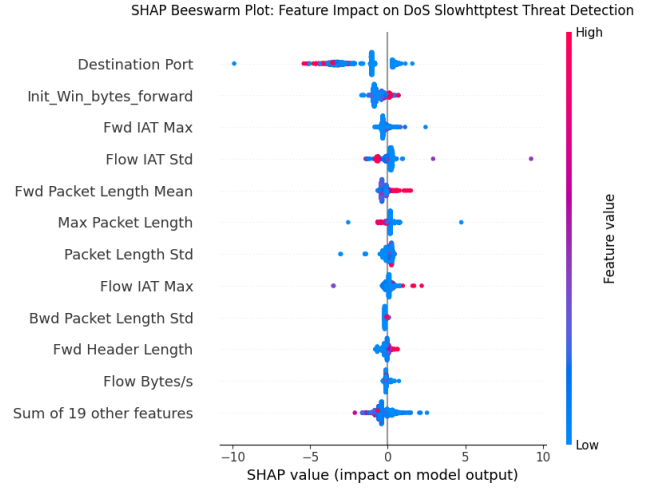


Fig. 13. SHAP Beeswarm for **DoS Slowhttptest**. Long-tail contributions on *Flow IAT Std* separate this class from faster DoS variants.

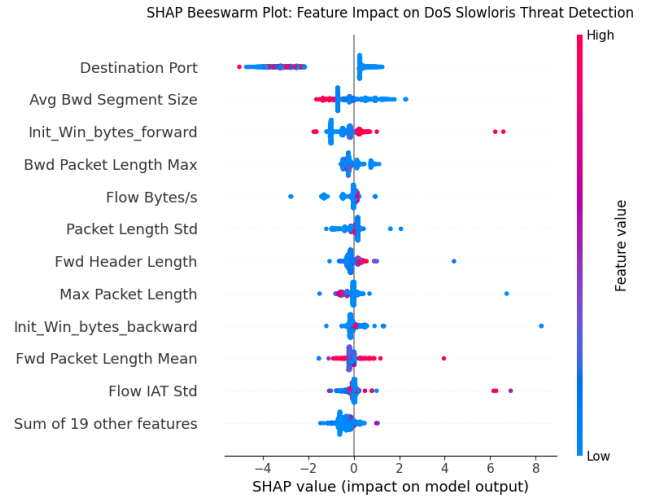


Fig. 14. SHAP Beeswarm for **DoS slowloris**. Low *Destination Port* entropy and *Init_Win_bytes_forward* are the primary drivers.

APPENDIX B RAW CONFUSION MATRIX

Fig. 23 reports the raw 5×5 confusion matrix computed on the reduced-class evaluation set used for the business prioritization study.

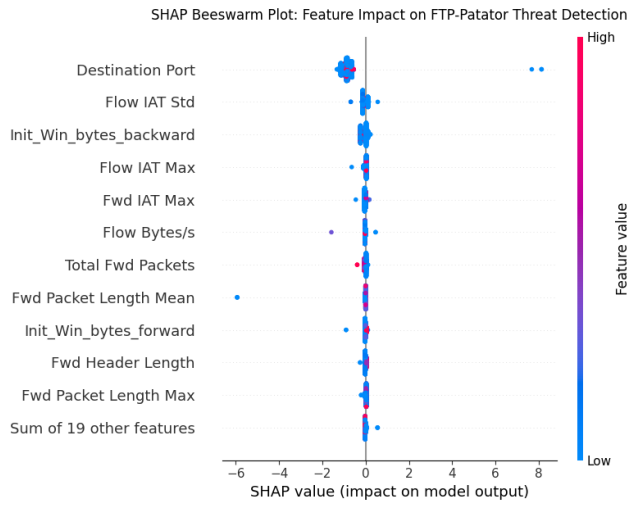


Fig. 15. SHAP Beeswarm for **FTP-Patator**. A very narrow feature contribution profile centered on *Destination Port* (port 21).

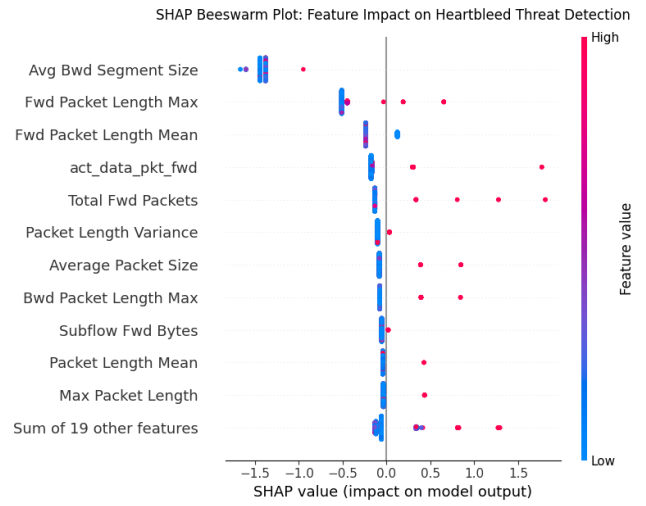


Fig. 17. SHAP Beeswarm for **Heartbleed**. *Avg Bwd Segment Size* and *Fwd Packet Length Max* are the dominant positive contributors.

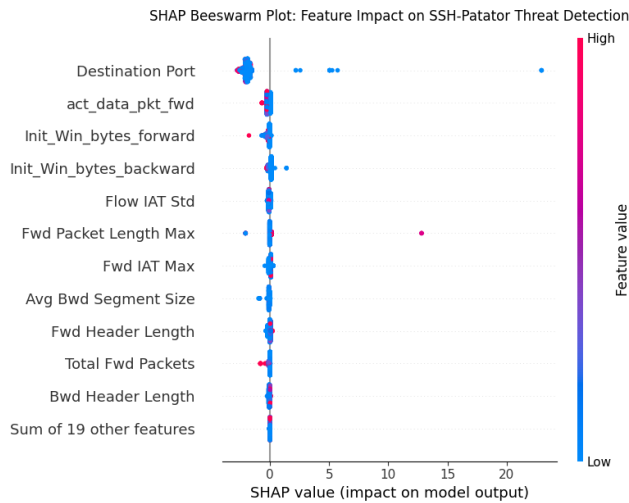


Fig. 16. SHAP Beeswarm for **SSH-Patator**. A single high-magnitude tail on *Destination Port* separates SSH-Patator alerts.

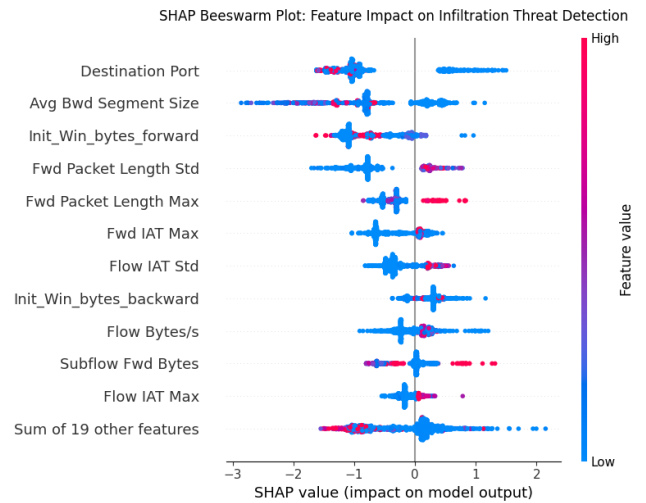


Fig. 18. SHAP Beeswarm for **Infiltration**. *Destination Port* and *Fwd Packet Length Std* show directional separation.

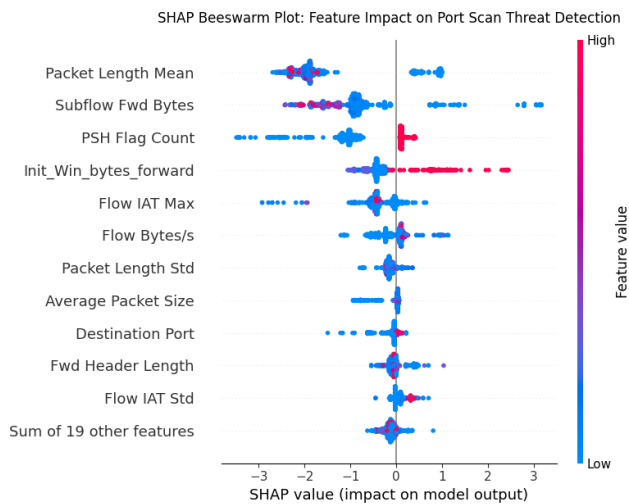


Fig. 19. SHAP Beeswarm for **PortScan**. *Packet Length Mean* and *PSH Flag Count* dominate; high *PSH Flag Count* is strongly indicative of sweep activity.

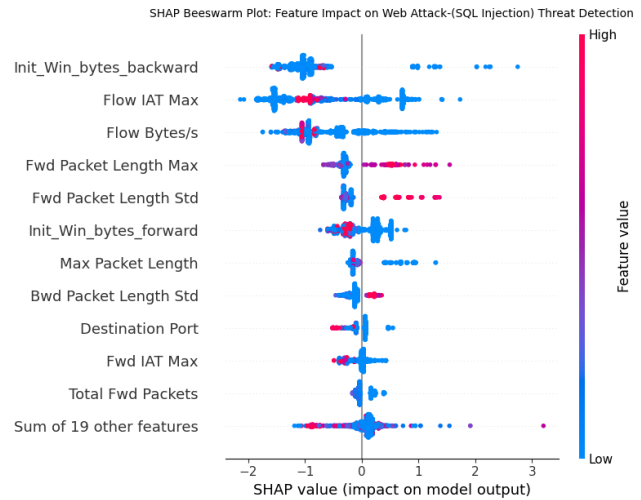


Fig. 21. SHAP Beeswarm for **Web Attack (SQL Injection)**. *Init_Win_bytes_backward* and *Flow IAT Max* dominate with narrow, high-value tails.

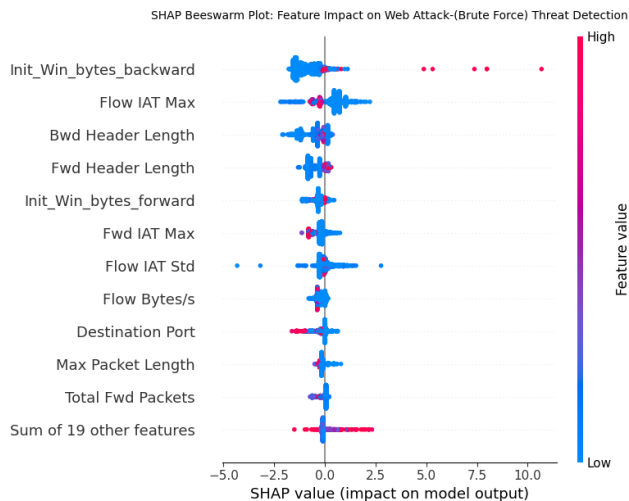


Fig. 20. SHAP Beeswarm for **Web Attack (Brute Force)**. *Init_Win_bytes_backward* shows a strong high-value tail driving positive classification.

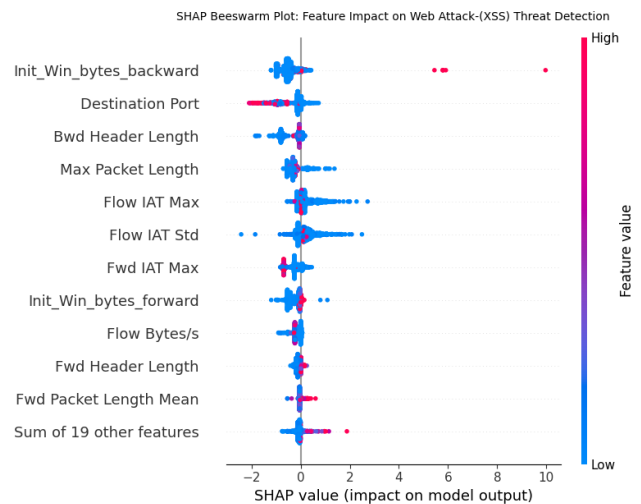


Fig. 22. SHAP Beeswarm for **Web Attack (XSS)**. *Init_Win_bytes_backward* again dominates, confirming its role as a browser-session discriminator.

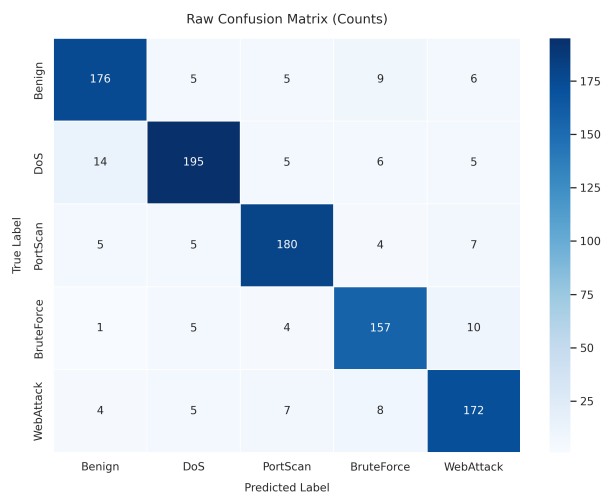


Fig. 23. Raw count Confusion Matrix on the reduced-class evaluation split.